INSURANCE CLAIM FRAUD DETECTION

SafeClaim · Machine Learning Course Project

U. Sindhuja · 2520080045
P. Bhagya Sri · 2520080024

Guide: Haritha

Department of AI & DS | KLH University
Academic Year 2026–27

Introduction

Vehicle insurance claims are the project's application domain.

Claim fraud detection uses claim and policy details to estimate whether a submitted claim may be fraudulent.

PROJECT OVERVIEW

A supervised classification problem

Two model classes: genuine or fraudulent

A saved XGBoost model sits behind a Flask web app

The result supports review; it does not decide a claim

Problem statement

Why this project is needed and who is affected.

Large claim volumes make it difficult to identify unusual patterns consistently from individual claim records.

01

INSURERS

Need to prioritize claims for further review.

02

CLAIM REVIEWERS

Must assess many claim and policy details.

03

POLICYHOLDERS

Honest customers can be affected by delays and fraud losses.

Project goal: provide a consistent first-pass model estimate for a reviewer to examine.

Motivation and objectives

A practical way to connect machine learning with an insurance use case.

WHY THIS PROJECT

Use a familiar, real-world classification problem to learn how data preparation, model selection, and a web interface fit together.

PROJECT OBJECTIVES

01 Prepare claim inputs for a saved model

02 Compare basic, grid-search, and Bayesian XGBoost

03 Build a Flask form for a single claim

04 Display prediction, probability, and review indicators

Existing and proposed system

The proposed app is a screening aid for a manual review workflow.

TYPICAL MANUAL REVIEW

Review each claim from its documents

Compare incident, policy, and vehicle details

Prioritize cases using reviewer judgment

SAFECLAIM DEMONSTRATION

Collect 15 claim attributes in a web form

Prepare a feature row and call XGBoost

Show classification, probability, and rule flags

Project boundary

The model suggests where to look; a person remains responsible for claim review.

Dataset and input data

fraud_oracle.csv contains vehicle-insurance claim records.

15,420	33	923	6.0%
claim records	columns, including target	fraud-labeled records	fraud class share

TARGET AND EXAMPLE INPUTS

Target **FraudFound_P**

Inputs Month · make · accident area · age · fault · vehicle value · prior claims · reports · witnesses · agent type · supplements

Dataset source URL was not included in the project files; add the original link before presenting.

Preprocessing and model inputs

The app prepares each submitted form as one model row.

01 Drop FraudFound_P from the predictor columns

02 Convert categories to integer category codes

03 Convert numeric fields and handle unknown values

04 Fill missing inputs with 0 and restore training column order

INPUT COVERAGE LIMIT

The dataset has 32 predictor columns; the form supplies 15. The app currently sets the other 17 fields to zero.

Algorithm and model

Bayesian-optimized XGBoost is the project's selected model.

XGBoost

A boosted-tree classifier used here to classify structured claim records.

MODEL SELECTION

Basic XGBoost — comparison baseline

Grid-search XGBoost — parameter search

Bayesian-optimized XGBoost — selected version

Saved artifact **insurance_fraud_model.json**

The project files do not document the final parameter values or tuning search space.

System architecture

One local web request passes through feature preparation and prediction.



Local files loaded at startup

fraud_oracle.csv

insurance_fraud_model.json

Methodology and workflow

The project connects offline model development to local inference.



Model development was done in Google Colab; the interface is served locally with Flask.

Technology stack and system modules

Local implementation; no database or cloud deployment is included.

TECHNOLOGY

Python · Flask

Pandas · XGBoost

HTML · CSS

CSV dataset · JSON model artifact

Google Colab · Visual Studio Code

APPLICATION MODULES

Claim-entry interface

Dataset and model loading

Feature encoding and alignment

Prediction and probability

Report with separate rule indicators

Database, authentication, cloud tools, and deployment are not part of the current app.

Application walkthrough

Use these four stages for the live demo.

01

HOME

Project overview and entry point

02

CLAIM FORM

Enter policyholder, vehicle, and incident details

03

MODEL STEP

Flask prepares features and calls XGBoost

04

RESULT REPORT

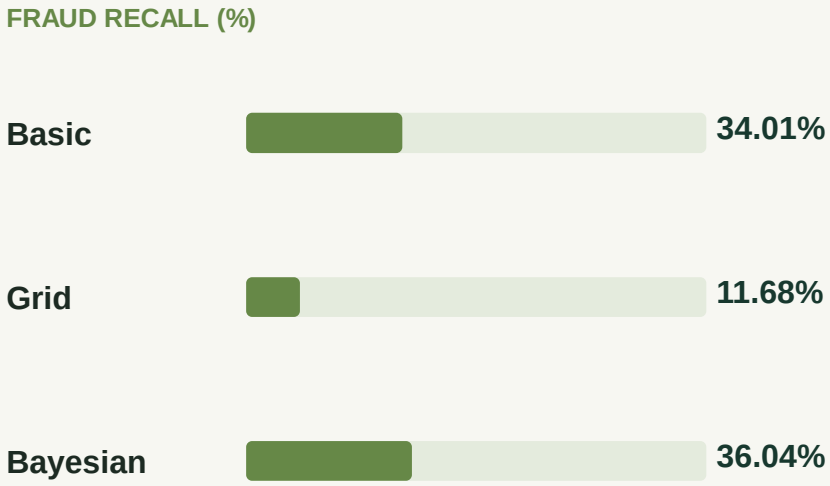
Read class, probability, and rule flags

This is a screen-flow overview; the project folder did not include captured screenshots.

Model results

The project’s performance page reports these comparison metrics.

Metric	Basic	Grid search	Bayesian XGBoost
Accuracy	95.10%	94.29%	**95.43%**
Precision	76.14%	92.00%	**82.56%**
Recall	34.01%	11.68%	**36.04%**
F1 score	47.02%	20.72%	**50.18%**
ROC-AUC	66.64%	55.80%	**67.76%**



Bayesian XGBoost has the highest reported accuracy, recall, F1, and ROC-AUC; Grid Search has the highest precision.

How to read a result

The model estimate and the demo review rules are separate outputs.

MODEL OUTPUT

Classification

Genuine / Fraudulent

Fraud probability

Risk band: <30% low · 30–<70% review · ≥70% high

DEMO RULE PRIORITY

2+ HIGH indicators → High review priority

1 HIGH or 2+ MEDIUM → Review suggested

Otherwise → Routine review

Rule indicators do not change the XGBoost prediction and do not prove fraud.

Benefits and applications

A first-pass screening aid for vehicle-claim review.

PROJECT BENEFITS

Demonstrates a complete ML-to-web workflow

Returns a consistent model output for each submitted claim

Keeps model probability separate from heuristic review flags

Supports classroom demonstration and further experimentation

POTENTIAL USE CASES

Vehicle-claim triage

Claims analytics coursework

Prototype for human-led review

Model comparison and explanation exercises

Potential applications only; the app is not validated or deployed for real insurance decisions.

Current limitations

Several gaps should be resolved before relying on the screening output.

LOW RECALL	36.04% recall means many positive cases may be missed.
INPUT MISMATCH	17 of 32 predictor fields are currently defaulted to zero.
METRIC TRACEABILITY	Training split, confusion-matrix counts, ROC plot, and tuning parameters are not supplied.
DEPLOYMENT	No database, authentication, access controls, or production hosting.
DATA SOURCE	The original online dataset URL is absent from the project files.

Future scope

Improvements that would make the project more useful and easier to validate.

01	ALIGN INPUTS	Use the same complete feature pipeline in training and inference.
02	IMPROVE RECALL	Compare class weighting, resampling, and thresholds on validation data.
03	REPRODUCE RESULTS	Document data source, split strategy, parameters, confusion matrix, and ROC curve.
04	EXTEND THE APP	Add secure user access, saved case history, and deployment monitoring.

Implementation steps

From dataset and model files to a local claim analysis.

01

Load CSV

Read claim data and identify target and predictors.

02

Prepare model

Compare XGBoost variants and save selected model.

03

Build interface

Create claim form and Flask prediction route.

04

Run locally

Load CSV and JSON model, submit a claim, show result.

Model training scripts and notebook are not in the supplied folder; the app uses the saved JSON model.

System modules and project status

Checklist module coverage, with absent components identified clearly.

User interface	**Implemented**	Flask / HTML claim form and results
Data collection	**Partial**	One claim entered manually; no live feed
Preprocessing	**Implemented**	Encode, align fields, fill absent values
Machine learning	**Implemented**	Saved XGBoost model
Database	**Not implemented**	CSV and JSON files only
Prediction and decision	**Implemented**	Class, probability and review flags
Visualization / reports	**Basic**	Result page and performance page
Authentication / security	**Not implemented**	No user login or role controls

Prediction code and output

The Flask route obtains both class and fraud probability from XGBoost.

```
# simplified from app.py

prediction = model.predict(X)[0]
probability = model.predict_proba(X)[0][1]

result = {
  'class': prediction,
  'fraud_probability': probability
}
```

RESULT PAGE DISPLAYS

Model classification

Estimated fraud probability

Low / review / high probability band

Separate rule-based review findings

Example output is described by fields only; no fabricated claim result is shown.

Technology and deployment context

The current deliverable is a local student prototype.

KNOWN FROM PROJECT

Python with Flask web framework

Pandas and XGBoost libraries

CSV data and JSON model file

Google Colab for model coding; VS Code for app work

NOT SPECIFIED OR NOT IMPLEMENTED

Hardware specifications are not recorded

No database configured

No cloud provider or hosted deployment

No production scaling or authentication

Run locally for a classroom demonstration; document machine requirements before deployment.

Responsible use and interpretation

A prediction is a review signal, not evidence of fraud.

MODEL

Learns patterns from historical labeled rows; output depends on training data.

REVIEWER

Checks documents and context before any claim decision.

POLICYHOLDER

Must be treated fairly; a false positive can lead to unnecessary review.

PROJECT LIMIT

Low recall and partial input coverage mean the demo can miss cases or be unreliable.

Next steps toward a stronger project

A practical order for improving quality and demonstrating the work.

01 Recover source dataset link and training notebook

02 Align all inference features with training columns

03 Validate on a documented holdout set; show confusion matrix and ROC

04 Capture real home, input, and results screens

05 Only then consider database, access controls, and hosting

Demo evidence checklist

Use actual application captures when presenting the live project.

01

HOME

Landing page

02

INPUT

Claim form

03

PROCESS

Flask prepares features and predicts

04

OUTPUT

Result page with class and probability

Screenshots were not included with the supplied project files. PROCESS is an internal step, not a separate screen.

CONCLUSION

SafeClaim links claim data, XGBoost, and a review interface.

The prototype demonstrates classification and reporting; improving input coverage and fraud recall are the next priorities.

THANK YOU · QUESTIONS

U. Sindhuja · P. Bhagya Sri · Guide: Haritha